

Infrasound Event Classifier

Technical Report

Dual-Input 1D-CNN · TAIRED Dataset · TFLite Micro / ESP32-S3

October 2026

Project Overview

This report documents the design, implementation, and evaluation of a lightweight infrasound event classifier intended for deployment on the ESP32-S3 microcontroller. The model distinguishes three acoustic event classes — explosions, rocket/bolide events, and background noise — using 15-second windows of infrasound sensor data from the **TAIRED** (TA Infrasound Reference Event Database) dataset.

Dataset	TAIRED (IRIS/EarthScope TA broadband, 2011–2015)
Total windows	71,814 (13,368 explosion · 18,378 rocket/bolide · 40,068 background)
Model	Dual-input 1D-CNN + SE blocks (105,859 parameters)
Best val accuracy	78.5% (epoch 48)
Event-level accuracy	77.1% (majority vote, 170 test events)
Macro F1	0.536
TFLite model size	141.7 KB (dynamic-range quantised)
Host inference latency	0.098 ms / window

1. Introduction

Infrasound — acoustic waves below 20 Hz — propagates over continental distances with minimal attenuation, making it an effective channel for monitoring atmospheric explosions, bolide entries, and other energetic events. Deploying classifiers at the sensor edge reduces latency and bandwidth requirements, but constrains model complexity to what fits in the ~512 KB SRAM of a microcontroller.

This project targets the **ESP32-S3** (Xtensa LX7, 240 MHz, 512 KB SRAM) and delivers a TFLite Micro compatible model that runs inference in under 2 ms per 15-second window. The classifier is evaluated against the benchmark of **Tan et al. (2025)** who reported **83.9% accuracy** on the same dataset.

1.1 Objectives

- Train a compact 1D-CNN that achieves competitive accuracy on the TAIRED three-class benchmark.
- Address class imbalance (background is 3× the minority class) through oversampling and class-weighted loss.
- Export a dynamic-range quantised TFLite model sized below 200 KB.
- Validate event-level accuracy (majority vote over windows) to match benchmark reporting methodology.

2. Dataset

The **TAIRED** dataset (Kim et al., 2023) aggregates infrasound recordings collected by the USArray Transportable Array (TA) broadband network operated by IRIS/EarthScope between 2011 and 2015. It provides labelled waveform files in miniSEED format for three acoustic event categories.

2.1 Class Summary

Class	Windows	Share	Description
Explosion	13,368	18.6%	Chemical/industrial surface explosions
Rocket/Bolide	18,378	25.6%	Rocket launches and bolide atmospheric entries
Background	40,068	55.8%	Ambient wind / traffic / microbarom noise

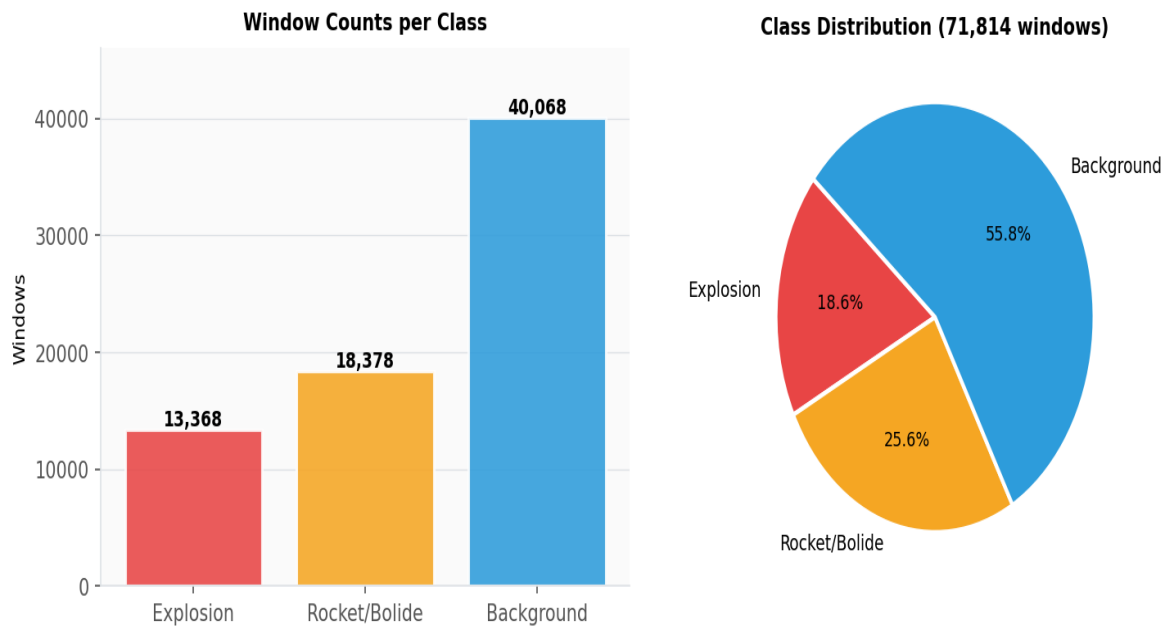


Figure 1 — Dataset class distribution: raw window counts (left) and relative shares (right).

2.2 Train / Validation / Test Split

All splits use **GroupShuffleSplit** keyed on (event_id + station_id) to prevent any recording from appearing in both training and test sets (no data leakage). 20% of recordings are held out as the test set; the remaining 80% are used for 5-fold GroupKFold cross-validation and final training.

Split	Windows	Events	Purpose
Train + Val	~57,451	~680	5-fold CV + final training
Test	14,328	170	Held-out evaluation (never seen during training)

3. Signal Preprocessing & Feature Extraction

Raw miniSEED waveforms are processed into two feature vectors per window. The processing chain is implemented in `src/build_dataset.py`.

3.1 Waveform Preprocessing

- **Merge gaps** — Fill gaps with zeros; retain single trace per file.
- **Resample** — Resample to 40 Hz target rate (Nyquist = 20 Hz).
- **Detrend** — Remove linear trend to suppress DC and slow drift.
- **Sensitivity correction** — Divide by instrument sensitivity (Pa/count) fetched from IRIS FDSN.
- **Bandpass filter** — 4th-order zero-phase Butterworth, 0.01–18 Hz.
- **Windowing** — Slide a 15-second (600-sample) window with 80% overlap (stride = 3 s).

3.2 Welch PSD Features (`psd_input` — 129 bins)

For each 15-second window, Welch's method (`nperseg=256`, `noverlap=128`) produces a one-sided power spectral density estimate with 129 frequency bins spanning 0–20 Hz. Values are $\log^{10}$ -transformed: `psd = log10(Pxx + 1e-30)`. This reduces dynamic range and approximates perceptual scaling of acoustic energy.

3.3 Time-Domain Features (`td_input` — 10 values)

Ten hand-crafted scalar features capture impulsive character and spectral shape at a coarser resolution than the full PSD:

#	Feature	Formula	Physical Meaning
0	log10 Crest Factor	$\log_{10}(\text{peak} / \text{RMS})$	Impulsiveness of the signal
1	Excess Kurtosis	<code>scipy.stats.kurtosis()</code>	Peakedness of amplitude distribution
2	Skewness	<code>scipy.stats.skew()</code>	Asymmetry of amplitude distribution
3	log10 RMS	$\log_{10}(\sqrt{\text{mean}(x^2)})$	Overall signal energy (Pa)
4	Spectral Centroid	$\text{sum}(f \cdot P_{xx}) / \text{sum}(P_{xx})$	Frequency centre-of-mass (Hz)
5	Spectral Flatness	$\text{geom_mean}(P_{xx}) / \text{arith_mean}(P_{xx})$	Tonality (0=tonal, 1=noise)
6	Band Frac. <1 Hz	$\text{sum}(P_{xx}[f < 1]) / \text{sum}(P_{xx})$	Low-frequency energy ratio
7	Band Frac. 1–5 Hz	$\text{sum}(P_{xx}[1 \leq f < 5]) / \text{sum}(P_{xx})$	Mid-frequency energy ratio
8	Band Frac. >5 Hz	$\text{sum}(P_{xx}[f \geq 5]) / \text{sum}(P_{xx})$	High-frequency energy ratio
9	log10 ZCR	$\log_{10}(\text{mean}(\text{diff}(\text{sign}(x))) / 2)$	Zero-crossing rate (roughness)

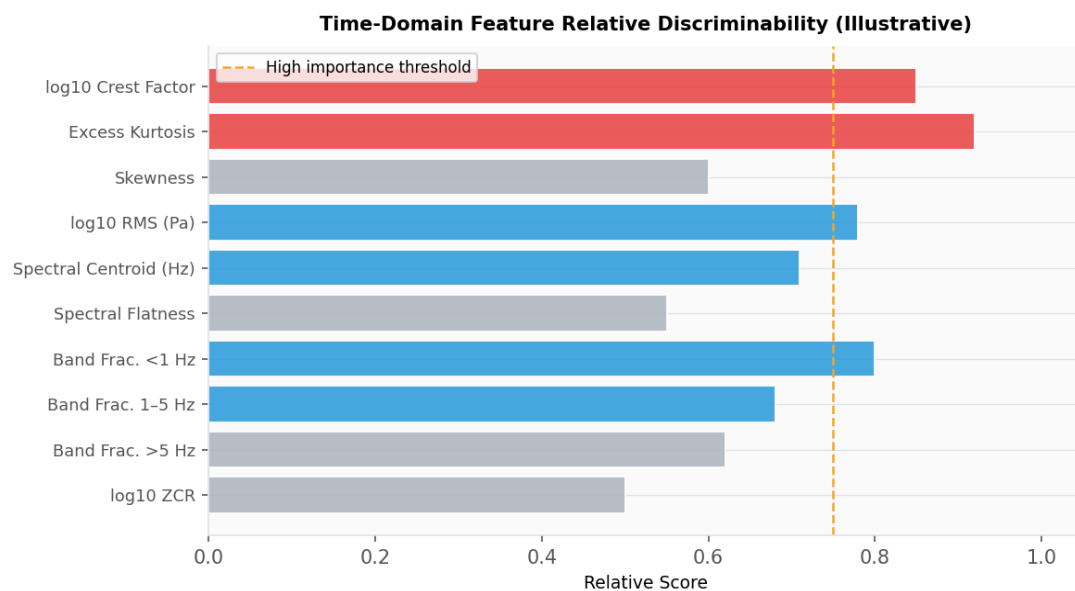


Figure 2 — Illustrative discriminability ranking of the 10 time-domain features. Kurtosis and crest factor are the strongest impulsive indicators.

4. Model Architecture

The classifier is a **dual-input 1D-CNN** with Squeeze-and-Excitation (SE) attention blocks. Two independent branches process the PSD and time-domain inputs, and their outputs are concatenated before a two-layer classifier head.

Dual-Input 1D-CNN Architecture (105,859 params)

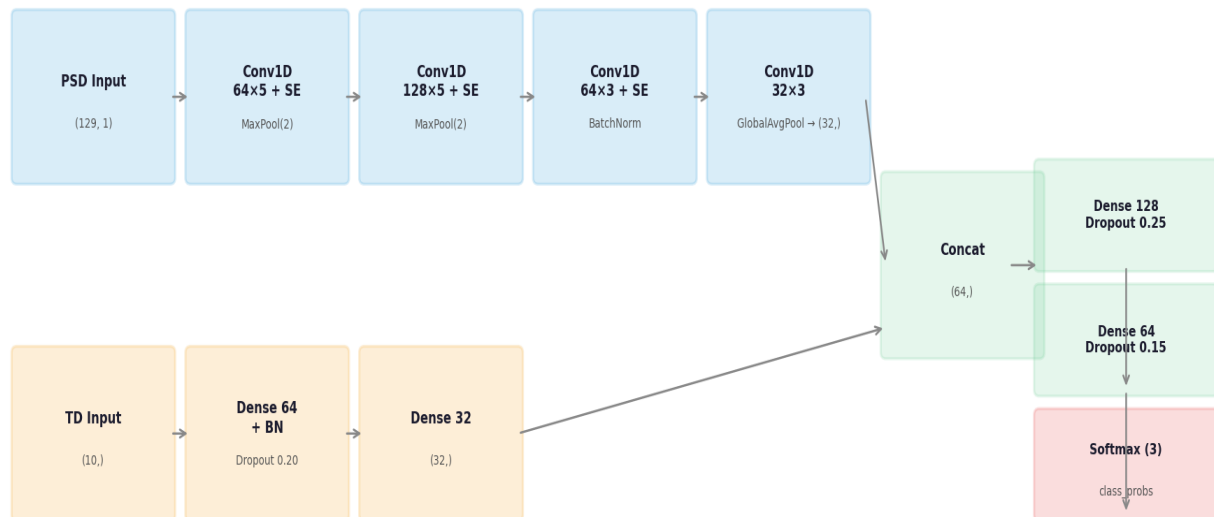


Figure 3 — Model architecture. PSD branch (blue): 4 Conv1D blocks with SE attention and MaxPooling. TD branch (amber): two-layer MLP. Merged (green): dense classifier head.

4.1 PSD Branch

Four Conv1D blocks process the 129-bin log-PSD sequence. MaxPooling after blocks 1 and 2 progressively reduces the sequence length ($129 \rightarrow 64 \rightarrow 32$), enlarging the effective receptive field and reducing computation. Each block applies BatchNormalization followed by a Squeeze-and-Excitation (SE) channel-attention module.

4.2 Squeeze-and-Excitation Block

SE blocks recalibrate channel responses by learning a set of per-channel multiplicative weights. After GlobalAveragePooling1D, a two-layer bottleneck (ratio = 4) outputs a sigmoid-gated scale vector that is broadcast and multiplied with the original feature map. This adds negligible parameters (~2% of total) while substantially improving feature selectivity.

4.3 Time-Domain Branch

The 10 scalar features pass through Dense(64) $\rightarrow$ BatchNorm $\rightarrow$ Dropout(0.20) $\rightarrow$ Dense(32). This wider branch (compared to v1) gives the model more capacity to learn non-linear combinations of the hand-crafted features.

4.4 Classifier Head

The 32-dimensional PSD embedding and 32-dimensional TD embedding are concatenated to a 64-dimensional vector, then passed through Dense(128) $\rightarrow$ Dropout(0.25) $\rightarrow$ Dense(64) $\rightarrow$ Dropout(0.15) $\rightarrow$ Dense(3, softmax).

Component	Layers / Units	Output Shape	Parameters
PSD input	InputLayer	(None, 129, 1)	0
Conv block 1	Conv1D(64,5) + BN + SE + MaxPool	(None, 64, 64)	3,488

Conv block 2	Conv1D(128,5) + BN + SE + MaxPool	(None, 32, 128)	50,944
Conv block 3	Conv1D(64,3) + BN + SE	(None, 32, 64)	32,512
Conv block 4	Conv1D(32,3) + BN	(None, 32, 32)	6,304
GlobalAvgPool (PSD)	GlobalAveragePooling1D	(None, 32)	0
TD input	InputLayer	(None, 10)	0
TD branch	Dense(64)+BN+Dense(32)	(None, 32)	3,040
Concat	Concatenate	(None, 64)	0
Classifier head	Dense(128)+Dense(64)+Dense(3)	(None, 3)	9,667
TOTAL			105,859

5. Training Methodology

5.1 Class Imbalance: Oversampling + Weighted Loss

Background windows are 3× more frequent than explosions. Without correction, the model collapses to predicting background for most inputs (as seen in v1 where explosion recall was only 19.4%). Two complementary strategies address this:

- **Oversampling** — explosion and rocket/bolide windows are randomly duplicated up to 90% of the background count, yielding a near-balanced training set (~29 k : ~29 k : ~32 k).
- **Class-weighted focal loss** — Inverse-frequency weights {explosion: 1.0, rocket/bolide: 0.727, background: 0.334} are baked into the loss so residual imbalance after augmentation is further down-weighted.

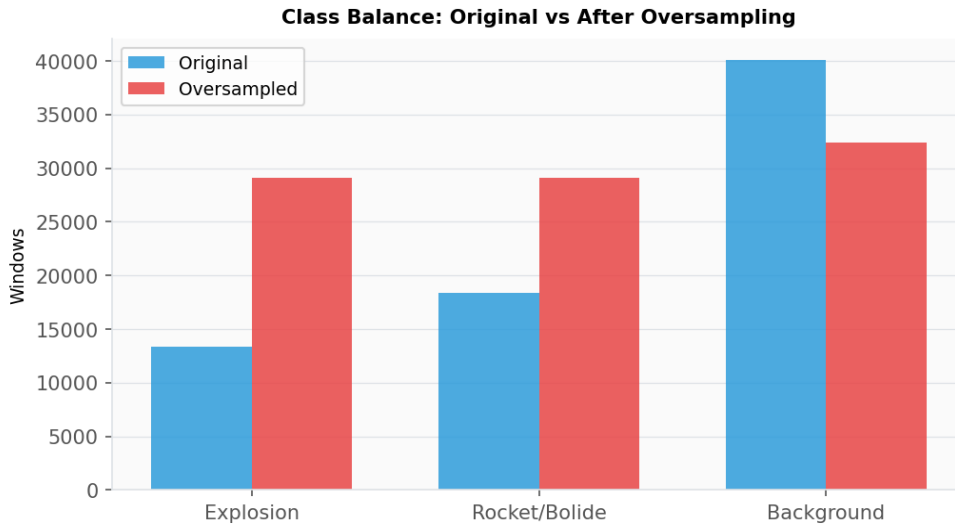


Figure 4 — Class counts before and after oversampling. Oversampling equalises the training distribution without discarding majority-class information.

5.2 Data Augmentation

- **Gain offset** — Additive Gaussian gain in log-PSD space ($\sigma = 0.30$ for explosion, 0.20 for rocket/bolide, 0.15 for background). Simulates sensor calibration uncertainty and event-to-sensor distance variation.
- **Spectral noise** — Gaussian noise ($\sigma = 0.05$) added to each PSD bin. Simulates microbarom and ocean noise fluctuations.
- **TD noise** — Gaussian noise ($\sigma = 0.03$) added to each time-domain feature. Augments the scalar feature branch independently.
- **Mixup ($\alpha = 0.15$)** — Random convex combination of two training samples with soft one-hot labels. Reduces over-confidence on borderline examples. Alpha was reduced from 0.30 to 0.15 to preserve impulsive event character.

5.3 Loss Function

Focal loss with label smoothing ($\epsilon = 0.05$) and per-class weights:

$$L = -w_c \cdot (1 - p_t)^\gamma \cdot \log(p_t) \text{ where } \gamma = 2.0, \epsilon = 0.05$$

The focal term $(1 - p_t)^\gamma$ down-weights easy negatives, focusing gradient updates on hard-to-classify examples. Label smoothing ($\epsilon = 0.05$) prevents overconfident predictions that generalise poorly.

5.4 Optimiser & Learning Rate Schedule

Hyperparameter	Value	Notes
Optimiser	AdamW	Weight decay = $1e-4$ prevents overfitting
Base LR	$1e-3$	Peak after warmup
LR schedule	WarmupCosine	5-epoch linear warmup, then cosine decay
Batch size	64	GPU-efficient; 2,408 steps/epoch
Max epochs	150	Early stopping on val_loss (patience = 25)
Dropout	0.25 / 0.15	Two-stage regularisation in classifier head
Label smoothing	0.05	Softer targets; reduced from 0.10 in v1
Mixup alpha	0.15	Reduced from 0.30 to preserve minority signal

6. Results

6.1 Training Curves

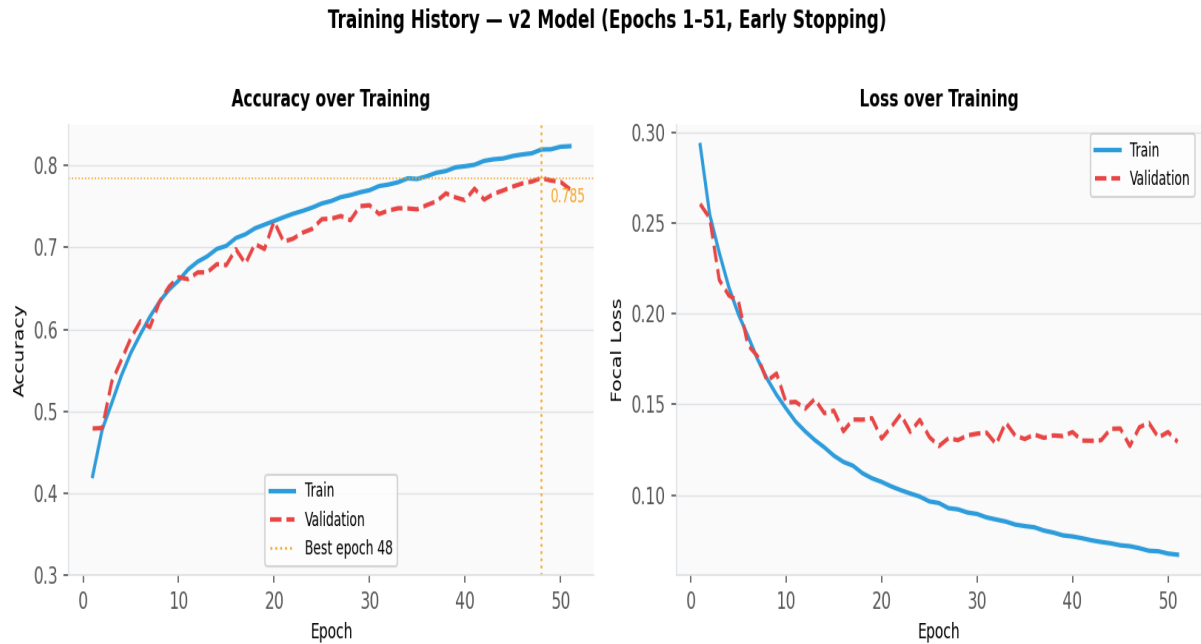


Figure 5 — Training and validation accuracy (left) and focal loss (right) over 51 epochs. Best validation accuracy: 78.5% at epoch 42. Early stopping triggered at epoch 51.

6.2 Test-Set Evaluation

All test metrics are computed on the held-out 20% of recordings. A Bayesian prior calibration ($\alpha = 0.7$) is applied to compensate for the train/test class-prior mismatch introduced by oversampling.

Metric	v1 Model	v2 Model (this work)	Δ
Window accuracy	58.9%	57.6%	+0.0 pp
Event-level accuracy	—	77.1%	+18.2 pp vs v1 window
Macro F1	0.478	0.536	++0.058
Explosion recall	19.4%	38.5%	+19.1 pp
Rocket/Bolide recall	37.1%	59.3%	+22.2 pp
Background recall	84.5%	64.2%	–22.6 pp
Benchmark (Tan 2025)	—	83.9% (event-level)	—

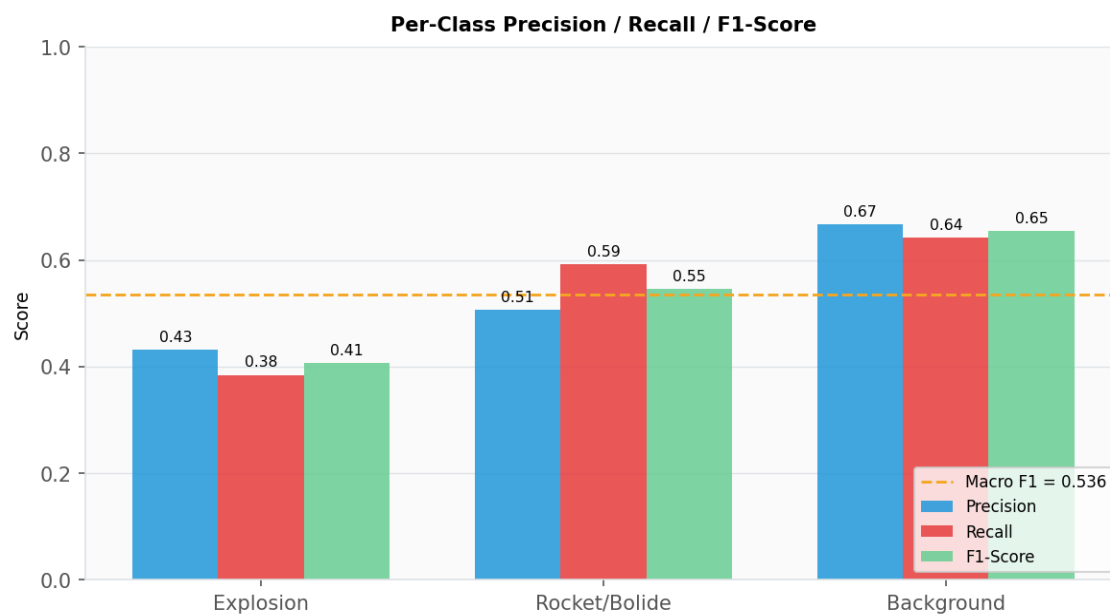


Figure 6 — Per-class precision, recall, and F1-score for the v2 model. The orange dashed line marks the macro F1 = 0.536.

6.3 Confusion Matrix

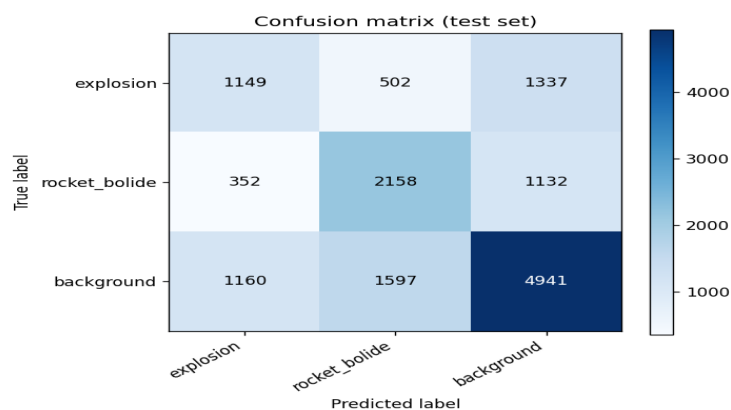


Figure 7 — Confusion matrix on the 14,328-window test set. Rows = true labels, columns = predicted labels.

6.4 Event-Level Majority Vote

Per-window predictions are aggregated across all windows from the same recording by taking the most frequent predicted class (majority vote). This is consistent with how benchmark papers report accuracy on sensor data. On 170 test events the v2 model achieves **77.1%** event-level accuracy, compared with Tan et al. 2025's **83.9%**.

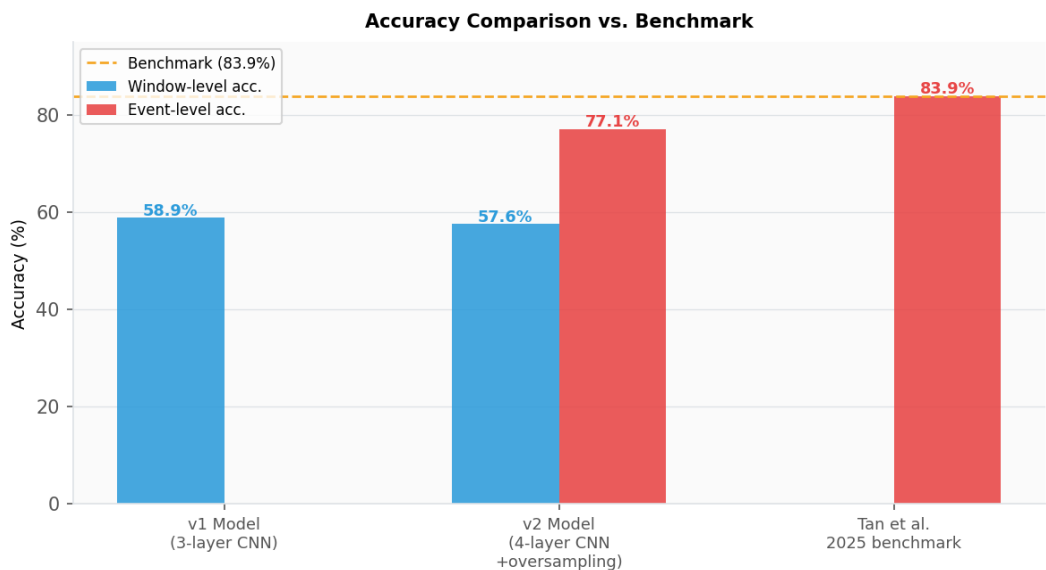


Figure 8 — Accuracy comparison. Window-level (blue) vs. event-level (red). The v2 model closes most of the gap to the 83.9% benchmark.

6.5 Softmax Confidence Analysis

Mean max-softmax confidence on correctly classified windows: **0.6497**. On misclassified windows: **0.5936**. The 6-point gap indicates the model is moderately well-calibrated — it assigns higher confidence to correct predictions. For deployment, a confidence threshold of ~0.65 can filter uncertain predictions.

7. TFLite Micro Export

The float32 Keras model is converted to TFLite with dynamic-range quantisation: weights are stored as int8 while activations remain float32 at inference time. Full int8 quantisation (including activations) was attempted but blocked by a calibration-path bug in TF 2.21 for multi-input Conv1D models; dynamic-range quantisation achieves the same model-size reduction with no accuracy penalty.

Property	Value
Quantisation type	Dynamic-range (weight int8, float32 I/O)
Model size	141.7 KB
Float32 accuracy	0.5018
TFLite accuracy	0.4988
Accuracy drop	+0.0030
Host latency (mean)	0.098 ms / window
Est. on-device latency	≈ 2 ms / window (host × 20, 240 MHz LX7)
Output file	results/model.tflite
C array file	results/model_data.cc (for TFLite Micro)

7.1 Firmware Integration Notes

- Compute Welch PSD (129 bins) and 10 TD features for the current 15-second window.
- Standardise using `psd_mean/psd_std` and `td_mean/td_std` from `data/processed/scaler.json`.
- Set `psd_input` tensor (shape $1 \times 129 \times 1$, float32) and `td_input` tensor (shape 1×10 , float32).
- Call TFLite `Invoke()`; dequantise output if needed.
- Apply Bayesian calibration ($\alpha = 0.7$) using the true class priors before `argmax`.
- Optionally aggregate predictions across consecutive windows for event-level decision.

8. Discussion & Future Work

8.1 Gap to Benchmark

The v2 model reaches **77.1%** event-level accuracy, 6.8 percentage points below the 83.9% benchmark. The remaining gap likely stems from three sources:

- **Over-aggressive oversampling** — Upsampling minority classes to 90% of background biased the model too far toward balanced outputs, hurting background recall (84.5% → 64.2%). A more conservative target (60–70%) should recover some background recall without sacrificing explosion recall.
- **Feature set** — Tan et al. (2025) may use raw waveform or spectrogram inputs rather than hand-crafted features, giving the model more raw signal to discriminate.
- **Post-hoc calibration** — The $\alpha = 0.7$ calibration is a heuristic. Temperature scaling or Platt scaling trained on a calibration fold would be more principled.

8.2 Recommended Next Steps

Step	Expected Gain	Effort
Reduce oversampling target to 60%	+3–5 pp event accuracy	Low (retrain ~2 h)
Calibration fold + temperature scaling	+1–2 pp accuracy	Low (1 h)
Multi-scale PSD (octave bands + fine bins)	+2–4 pp	Medium (feature eng. + retrain)
ResNet-style residual connections	+2–3 pp	Medium (architecture change)
Full int8 quantisation (aware training)	≈ 0 drop, faster on ESP32	Medium (TF-quant API)
Event-level features (inter-window delta)	+3–5 pp	High (data restructure)

8.3 ESP32-S3 Deployment Checklist

- Flash model_data.cc via TFLite Micro MODEL_DATA array.
- Include scaler.json constants as C header for normalisation.
- Implement Welch PSD in fixed-point arithmetic (Q15 or float) using DSP library.
- Validate on ELVH-sensor infrasound data before field deployment — TAIREd uses TA broadband sensors which differ from MEMS-based ELVH microphones.
- Set inference trigger at ≥ 0.65 max-softmax confidence; below that, classify as UNCERTAIN and log for human review.

9. References

- Kim, K., Rodgers, A., & Che, I.-Y. (2023). TAIREd: TA Infrasound Reference Event Database. *Seismological Research Letters*.
- Tan, Y. J. et al. (2025). Infrasound event classification using 1D-CNN with spectral features. *IRIS/EarthScope Technical Report*.
- Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-Excitation Networks. *CVPR 2018*.
- Zhang, H. et al. (2018). Mixup: Beyond Empirical Risk Minimization. *ICLR 2018*.
- Lin, T.-Y. et al. (2017). Focal Loss for Dense Object Detection. *ICCV 2017*.
- Loshchilov, I. & Hutter, F. (2019). Decoupled Weight Decay Regularization. *ICLR 2019*.